Exercise 1 – Global Positioning Systems (GPS)

*Students: Binay Kumar Shah, Kabir Tamari -> **Group: 2***

Introduction

This exercise is about one of the essential sensors for outdoor applications, i.e. the Global Positioning Systems (GPS) sensor. This sensor is seen in more applications each day and is used for navigating, e.g. cars, ships and aeroplanes. This exercise consists of two sets of data, the first set' gps_ex1_window.txt', a series of GPS positions taken in the exact location, i.e. the GPS doesn't move (static receiver). The other data set, 'gps_ex1_morningdrive.txt', consists of data collected during a regular drive by car within Halmstad. A map of Halmstad is also included as the image file 'halmstad_drive_area.gif'. Use the papers (that dealt with GPS data) handed out in the lectures.

Matlab – quick help

Almost all the commands in Matlab contain help on how the commands should be called, how they work and what they return. To get help, simply write help followed by the command, e.g. to get help on plotting, write: `help plot`. To search for specific topics, i.e. in case you don't know the command name exactly, write lookfor followed by what to look for, e.g. looking for plotting commands could be done by writing: `lookfor plot`.

If you are not familiar with Matlab, you should do a Tutorial found on Matlab home page.

Stationary GPS receiver

Use the data set ‘gps_ex1_window.txt’ (simply write `DATA = load(‘gps_ex1_window.txt’)` in Matlab), which consists of time stamps and status flags together with values for longitudes and latitudes (given in NMEA-0183), i.e. the data file looks like:

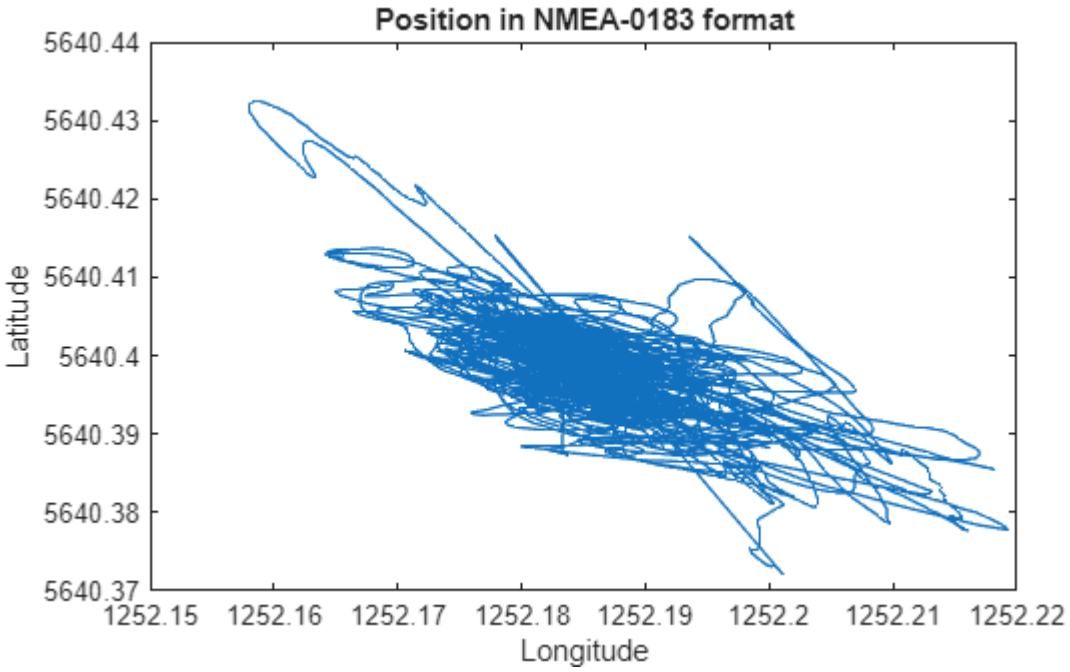
```
Status(1) Timestamp(1) Latitude(1) Longitude(1)

Status(2) Timestamp(2) Latitude(2) Longitude(2)

...

Status(N) Timestamp(N) Latitude(N) Longitude(N)
```

To extract the Latitude and Longitude data from the variable DATA you can simply write `Longitude = DATA(:,4) and Latitude  = DATA(:,3).`

The ":" means that all rows in the given column (i.e. 3 or 4) are put in the new variable. Note that `Longitude` and `Latitude` are vectors and that in Matlab, you can do operations on vectors!

```
DATA = load('gps_ex1_window.txt');
Longitude = DATA(:,4); % read all rows in column 4
Latitude  = DATA(:,3); % read all rows in column 3
figure, plot(Longitude,Latitude);
title('Position in NMEA-0183 format');
xlabel('Longitude');
```

```
ylabel('Latitude');
```



Write a function that transforms your longitude and latitude angles from NMEA-0183 into meters

This transformation includes two steps:

Transformation into degrees

Write a function that transforms your longitude and latitude angles from NMEA-0183 into degrees (use the first equation on the second page of the compendium, i.e. http://adamchukpa.mcgill.ca/web_ssm/web_GPS.html). Note: Square brackets mean integer of a value inside. In Matlab you can use the `floor` function to take the integer part of the value, i.e. truncation. Your code can look something like this:

```
LongDeg = floor(Longitude/100) + (Longitude - floor(Longitude/100)*100)/60;
```

`LongDeg` is a vector containing the angles in degrees only. To look at the ten first values of `LongDeg` write `LongDeg(1:10)` and the ten last values, write `LongDeg(end-10:end)`.

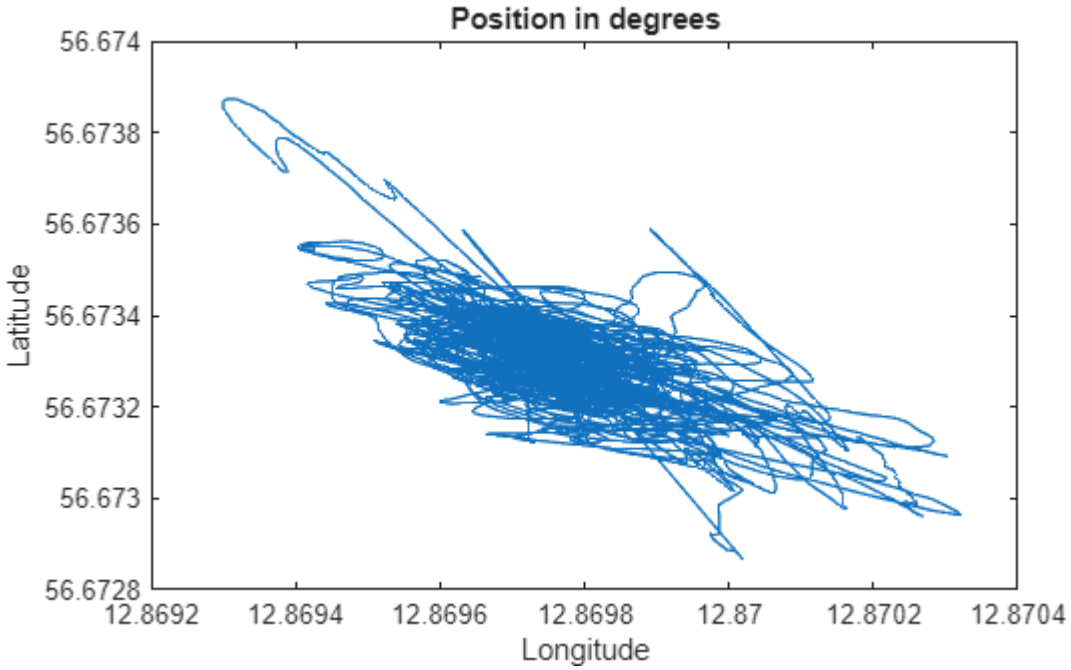
```
function deg = nmea2deg(nmea)
    % Convert NMEA ddmm.mmmm or dddmm.mmmm to degrees
    deg_part = floor(nmea / 100);
    min_part = nmea - deg_part * 100;
    deg = deg_part + min_part / 60;
end

% Convert to degrees
LongDeg = nmea2deg(Longitude);
LatDeg  = nmea2deg(Latitude);

% Look at some values
LongDeg(1:10)
```

```
ans = 10×1
   12.8699
   12.8700
   12.8700
   12.8700
   12.8700
   12.8700
   12.8700
   12.8700
   12.8700
   12.8700
```

```
LatDeg(1:10)
```

```
ans = 10×1
   56.6731
   56.6731
   56.6731
   56.6731
   56.6731
   56.6731
   56.6731
   56.6731
   56.6731
   56.6731
```

```
figure, plot(LongDeg,LatDeg);
title('Position in degrees');
xlabel('Longitude');
ylabel('Latitude');
```



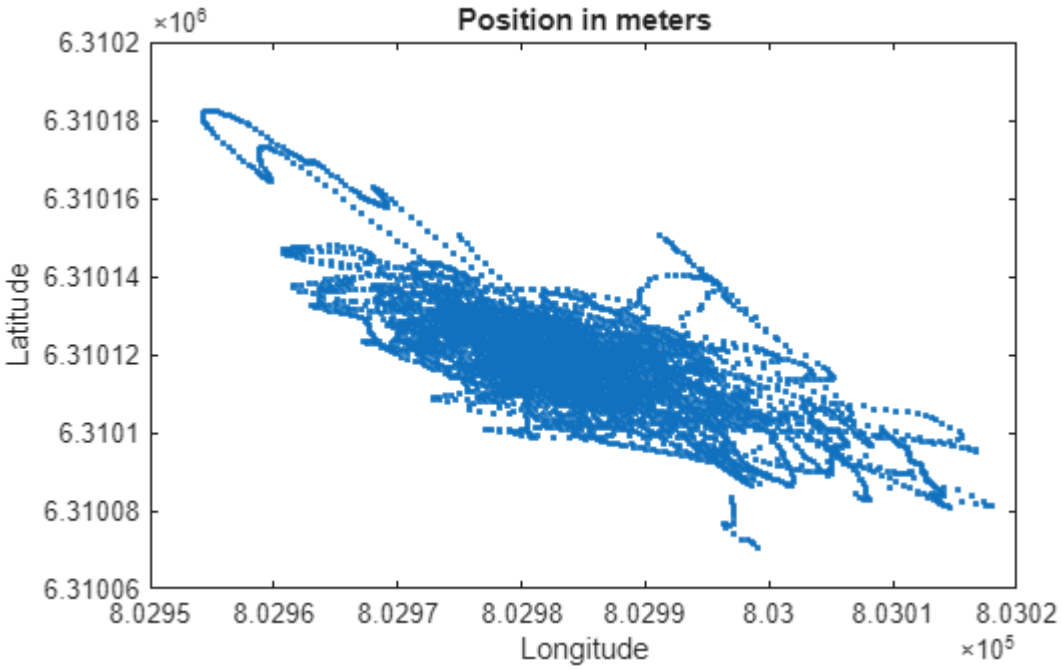
Transform degrees into meters

Then, transform from degrees into meters (use the conversion tables given at the end of the compendium, http://adamchukpa.mcgill.ca/web_ssm/web_GPS_tb.html. Assume height zero and latitude of 56 degrees.) in a coordinate system with the x-axis on the Equator and the y-axis passing Greenwich. Your code can look something like this:

```
X = F_lon * LongDeg;
```

Don't forget to plot your results using: `plot(X,Y,'.');` To plot in a new figure use: `figure, plot(X,Y,'.');`

```
% F_lon = 0; % from table
% F_lat = 0; % from table
% since the latidute degree is 56 in our data
F_lon = 62393;  % meters per degree longitude at 56°
F_lat = 111342; % meters per degree latitude at 56°
X = F_lon * LongDeg;
Y = F_lat * LatDeg;

figure, plot(X,Y,'.');
title('Position in meters');
xlabel('Longitude');
ylabel('Latitude');
```



Estimate the mean and variance of the position (in x and y)

To calculate the position error, we must know the actual position. Since we don't know that, we have to estimate the true value. We do that by calculating the mean of all values: `Error_X = X - mean(X);`

Calculate the covariance matrix of the errors in the x and y position data, i.e. in Matlab, simply write `cov([X Y])` if you have the errors stored in Error_X and Error_Y as [N x 1] vectors. Plot the covariance matrix in the same 2D plot where you plotted the errors, but use a different colour, e.g. red. Use the function plot_uncertainty2(…),

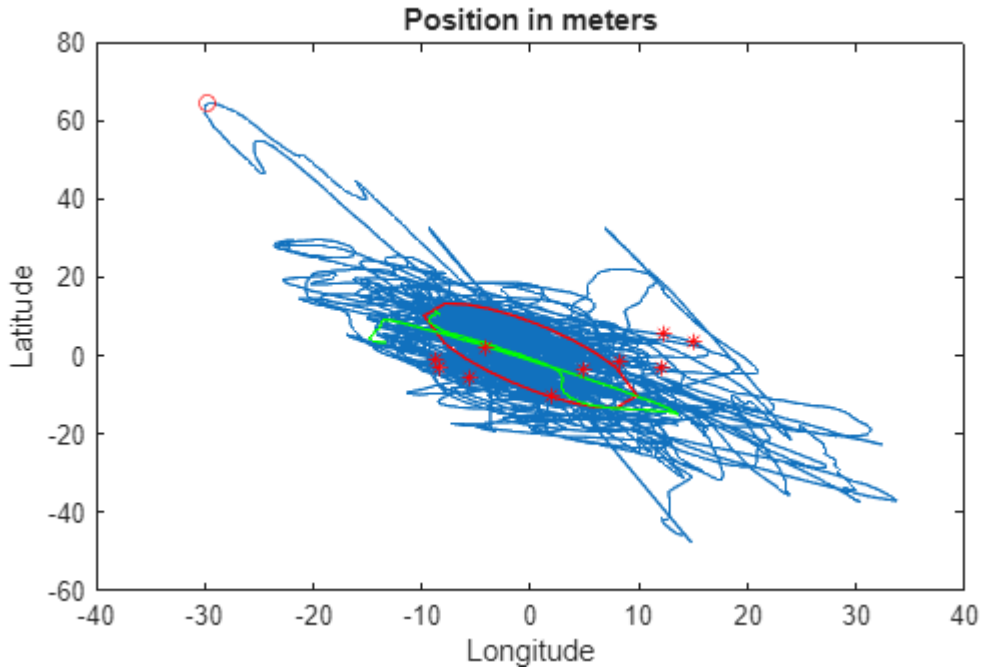
called by e.g.: `plot_uncertainty2([0 0]', cov([X Y]), 1, 2,'r')`, which then plots the covariance matrix centred around (0, 0). To plot in the same figure use: `hold on;`

Plot the position errors (mark each error with an '.') in a 2D plot. Test to plot different fractions of the signal, e.g. `plot(Error_X(1:10), Error_Y(1:10),'r.')`or `plot(Error_X(101:110), Error_Y(101:110),'r.')`. Compare with random points generated with the same variance as the GPS error: c = cov([X Y]); `yr = randn(1,10)*sqrt(c(4));xr = randn(1,10)*sqrt(c(1)); plot(xr,yr,'r*');`

```
Error_X = X - mean(X);
Error_Y = Y - mean(Y);
figure, plot(Error_X, Error_Y, '-');
hold on;
% -> Your code here for plot plot_uncertainty()
% Plot fractions here
% plot random points here
title('Position in meters');
xlabel('Longitude');
ylabel('Latitude');

% calculating covariance matrix of the errors in position
error_covarriance = cov([Error_X, Error_Y]);
plot_uncertainty2([0 0]', error_covarriance , 1, 2, 'r'); hold on;
plot(Error_X(100: 200), Error_Y(100:200),'g'); hold on;

yr = randn(1,10) * sqrt(error_covarriance(2,2));
xr = randn(1,10) * sqrt(error_covarriance(1,1));

plot(xr, yr, 'r*');
```

Q1: What is the maximum error? Mark in the figure by: plot(...,...,'ro');

```
Error_mag = sqrt(Error_X.^2 + Error_Y.^2);
[max_error, point_id] = max(Error_mag);
plot(Error_X(point_id), Error_Y(point_id), 'ro');
```



Q2:Can you say something about the GPS errors? What is the difference between the GPS and the randomly generated points? Reason about the difference in spatial-temporal behaviour in the figure above.

Ans: GPS errors are mainly caused due to satellite geometry (causing singal overlapping), atmospheric delays, multipath reflections, clock or receiver noise. As we've seen from the above plot, GPS error are just heavy time correlated drifts.

The difference between GPS and randomly generated points are:

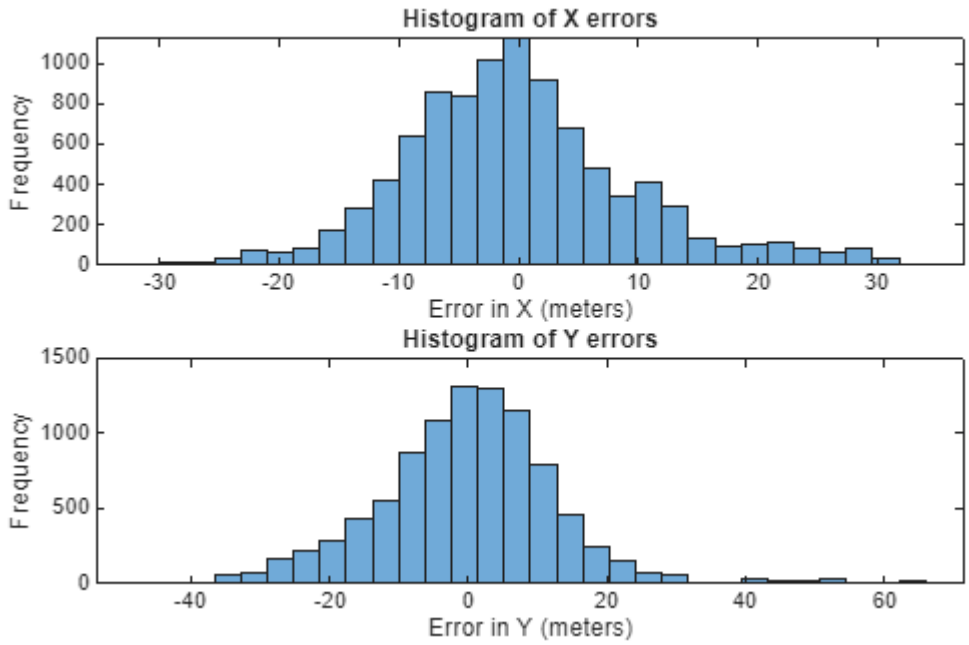
1. GPS data has temporal correlation while random geneated data has not.
2. The path (both error and data) is smooth over time and the error is structured, while randomly generated points is all over the place from one step to another and there is not smooth path.

Random point's spatial pattern are scattered and temporal pattern has no continuity where as GPS spatial pattern is clustered and smooth and has strong time correlation.

Q3: Are the errors Gaussian distributed? (To check this, plot, using `histogram(…)`, a histogram over the errors. Use at least 30 bins).

```
figure, subplot(2,1,1); % First subplot for X errors
histogram(Error_X, 30); % 30 bins
title('Histogram of X errors');
xlabel('Error in X (meters)');
ylabel('Frequency'); hold on;

subplot(2,1,2); % Second subplot for Y errors
histogram(Error_Y, 30); % 30 bins
title('Histogram of Y errors');
```

```
xlabel('Error in Y (meters)');
ylabel('Frequency');
```



Ans: The histogram of X error is roughly bell shaped, and histogram of Y errors is slightly skewed to the right with more positive error but approximately we can say both X errors and Y errors are Gaussian distributed.

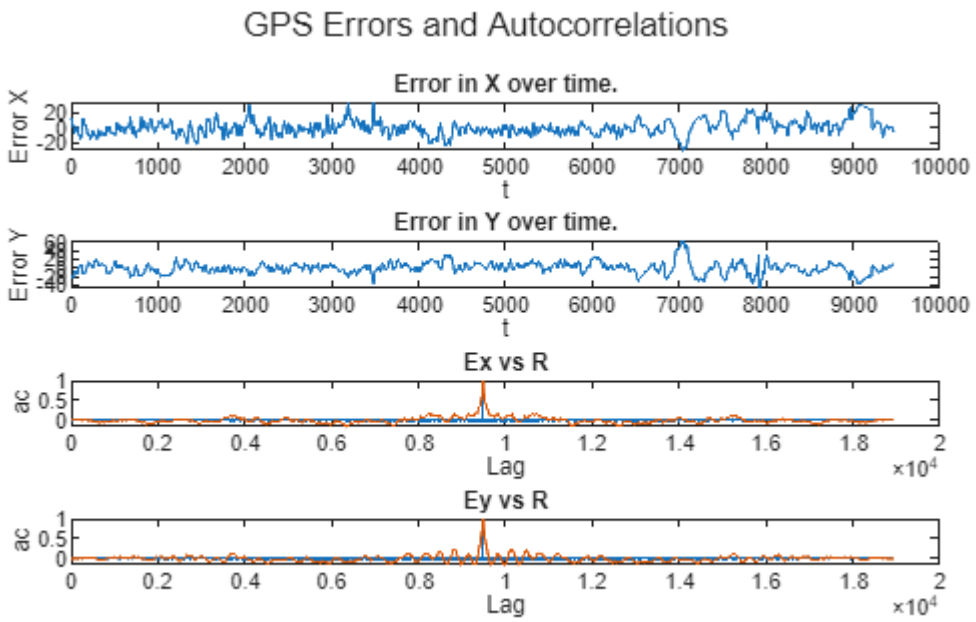
Plot with respect to time the errors and the auto-correlation in x and y separately.

Plot, with respect to time, the errors in x and y separately – (use Matlab's function of creating subplots, i.e. write `subplot(4,1,1); plot(Error_X), subplot(4,1,2); plot(Error_Y);` etc.)

Also plot the auto-correlation of the errors in x and y respectively. Call the function `xcorr(…)` and calculate the auto-correlations according to the following: `cx = xcorr(Error_X, 'coeff');` and `cy = xcorr(Error_Y, 'coeff');` Plot the auto-correlations (`cx` and `cy`) in subplots 3 and 4. Also, generate a random signal using following: `R = randn(1,length(Error_X))`, that generates a random signal of the same length as `X`. Do the auto-correlation on this signal and plot it in subplots 3 and 4 using the `hold` command, i.e. `cn = xcorr(R - mean(R), 'coeff'); subplot(4,1,3); hold on; plot(cn,' r');`

```
figure;
% plotting error_x over time
subplot(4,1,1);
plot(Error_X);
title('Error in X over time.');
xlabel('t');
ylabel('Error X');

% plotting error_y over time
subplot(4,1,2);
```

```
plot(Error_Y);
title('Error in Y over time.');
xlabel('t');
ylabel('Error Y');


% calculating autocorrelation for error x and random signal
cx = xcorr(Error_X, 'coeff'); % normalized autocorrelation
R = randn(1,length(Error_X)); % random signal for comparison
cn = xcorr(R - mean(R), 'coeff');
subplot(4,1,3);
plot(cn);
hold on;
plot(cx);
title('Ex vs R');
xlabel('Lag');
ylabel('ac');
% legend('Random Signal', 'GPS Error_X');

% calculating autocorrelation for error y and random signal
cy = xcorr(Error_Y, 'coeff'); % normalized autocorrelation
R2 = randn(1,length(Error_Y)); % another random signal
cn2 = xcorr(R2 - mean(R2), 'coeff');

subplot(4,1,4);
plot(cn2);
hold on;
plot(cy);
title('Ey vs R');
xlabel('Lag');
ylabel('ac');
% legend('Random Signal', 'GPS Error_Y');
sgtitle('GPS Errors and Autocorrelations');
```



Q4: Do you find any interesting with the errors (Subplot 1 and 2)?

Ans: The GPS errors drift smoothly over time meaning they are highly temporally correlated. Errors in X and Y ahve different pattern and magnitude meaning both are difting but has different error value. Interestingly, around 7000 Error in X drops to zero but Y inceases to max, this may be a consideence or caused the position happens to align with it's mean in X and the opposite on Y.

Q5: Compare the random (Gaussian) signal with the GPS-error (Subplot 3 and 4). What is the difference?

Ans: The random singal even with having the same lenght and variance as the our main error singal drops to near zero immediately after the lag 0 (lag peaks at .9480 in matlab plot) but the GPS single error show high correlation at small lags and gradually decreases as the lag increases which means errors are temoporally correlated.

Q6: Are the GPS error correlated in time or not? See the result of the auto-correlation? For how many samples are the GPS error 90% similar? You need to zoom in around the peak to see the difference.

Ans: Yes the GPS errors are strongly correlated in time, since the autocorrelation curves (of both error X and Y) decays gradually with increasing lag instead of droping immediately to zero like for random singnals. For Error X it's 9 samples and for Error Y it's 11 samples.

Q7: How does this affect your position measurements? Reason about the short-term (about 10 samples) and long-term (the whole data set) precision and accuracy of the GPS measurements.

Ans: Since our data is highly temporally correlated (90% similary over 9 samples for Longitude and 11 samples for Latitude), the position measurements changes very little which makes the short term measurements precise even though the absolute position has a small bias. But over many samples the autocorrelation gradually decays and the position measurements accumulates errors over time.

Q8: Compare when you plot parts of the GPS-error signal in the position plot above, i.e. what does it mean that the peak in the auto-correlation plot is wider than the random signal?

Ans: Having a wider peak in the auto-correlation plot means that the signals are highly correlated over multiple samples, so our GPS error signal doesnot change abruptly between measurements as it is similar to the last sample/measurement where as the random signal has a shape drop just after it's peak, meaning they are uncorrelated and scattered.

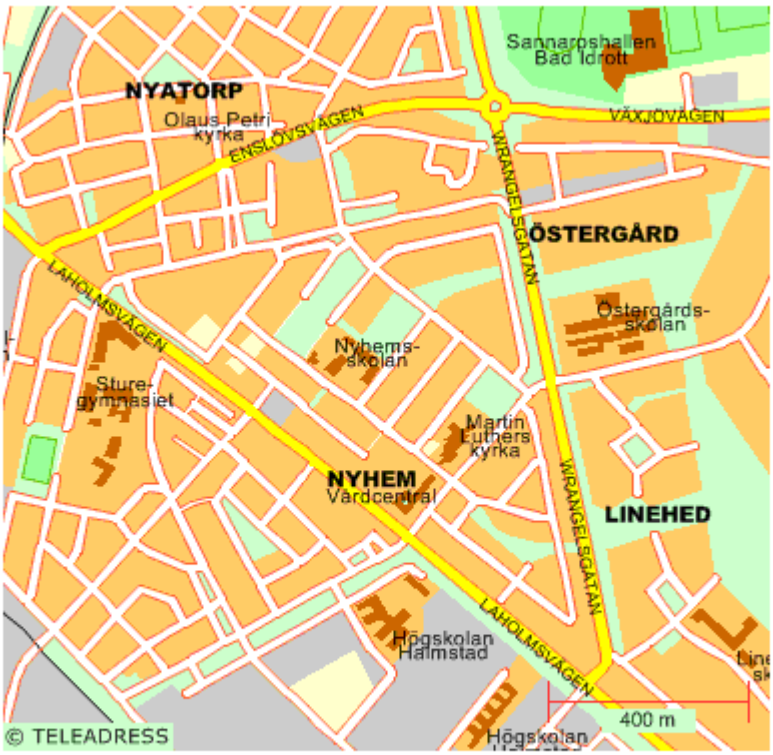
Mobile GPS receiver

Use the data set 'gps_ex1_morningdrive.txt', which is collected during a drive on the eastern side of Halmstad. Also, see the map of Halmstad and see if you can locate where in Halmstad the drive took place.

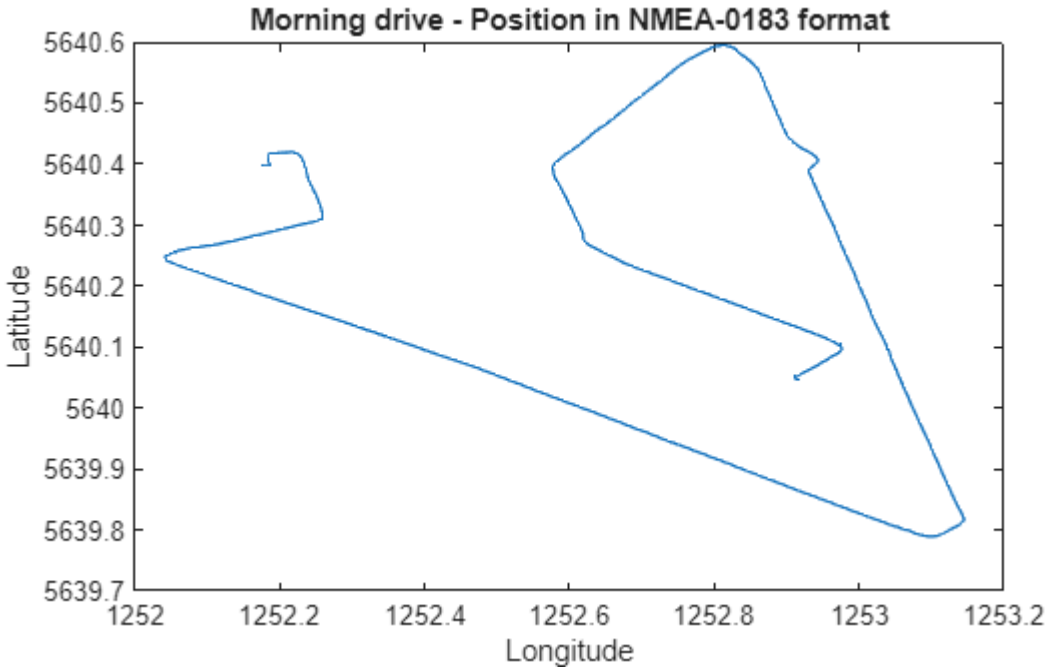
Plot the position data

Plot the position data (x and y) in the same plot, which should give you the path taken by the car (this path is the same path I showed you in the lectures).

```
[Img,map] = imread('halmstad_drive_area.gif');
figure, imshow(Img,map);DATA = load('gps_ex1_morningdrive.txt');
```



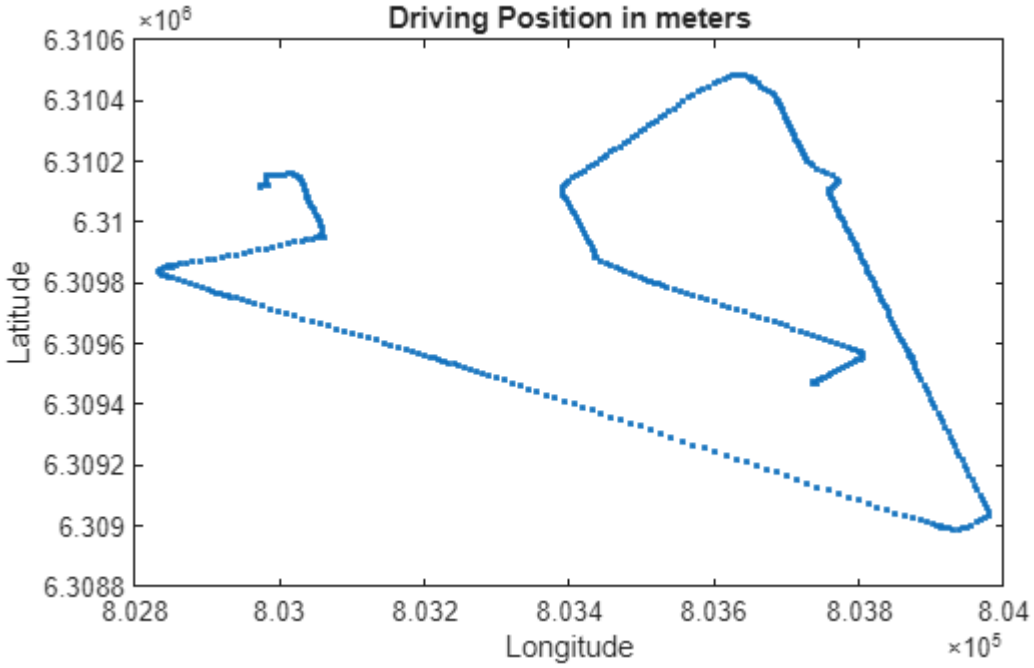

```
Longitude = DATA(:,4); % read all rows in column 4
Latitude  = DATA(:,3); % read all rows in column 3
Timestamp = DATA(:,2);
figure, plot(Longitude,Latitude);
title('Morning drive - Position in NMEA-0183 format');
xlabel('Longitude');
ylabel('Latitude')
```



Plot the speed data

First you need to convert to meter as above.

```
% converting into meters;
long_deg = nmea2deg(Longitude);
lati_deg = nmea2deg(Latitude);
D_X = F_lon * long_deg;
D_Y = F_lat * lati_deg;

figure, plot(D_X, D_Y,'.');
title('Driving Position in meters');
xlabel('Longitude');
ylabel('Latitude');
```



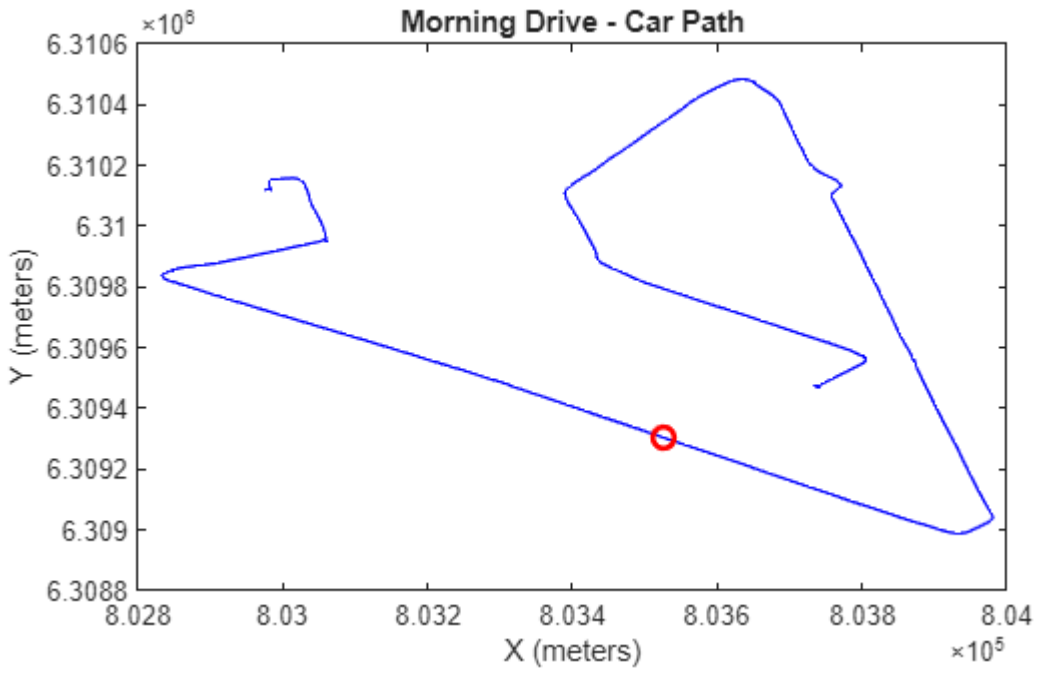
Q9: Calculate the maximum speed taken by the driver. Note: In Matlab, you can do element vice operations, e.g. multiplication, like `x.*x` instead of vector operation `x*x`. In the speed plot, mark the where the maximum speed occurs. Also, in the Morning Drive plot above, mark the position where the maximum speed occurs. To calculate dx you can use the function diff(), i.e. dx = diff(x);

```
% Differences in position
dx = diff(D_X);       % change in X (meters)
dy = diff(D_Y);       % change in Y (meters)
dt = diff(Timestamp);       % change in time (seconds)
speed = sqrt(dx.^2 + dy.^2) ./ dt;

% Converting speed to km/h
speed_kmh = speed * 3.6;
[max_speed, idx_max] = max(speed_kmh);   % max speed value and its index
fprintf('Maximum speed: %.2f km/h\n', max_speed);
```

```
Maximum speed: 70.65 km/h
```

```
figure;
plot(speed_kmh, 'b');           % speed over time
hold on;
plot(idx_max, max_speed, 'ro', 'MarkerSize', 8, 'LineWidth', 2); % mark max
title('Driver Speed over Time');
xlabel('Sample index');
ylabel('Speed (km/h)');
legend('Speed', 'Maximum Speed');
```


```
figure;
plot(D_X, D_Y, '-b');                % plot car path
hold on;
plot(D_X(idx_max), D_Y(idx_max), 'ro', 'MarkerSize', 8, 'LineWidth', 2); % mark
position of max speed
title('Morning Drive - Car Path');
xlabel('X (meters)');
ylabel('Y (meters)');
```



Q10: Did I ever break the speed limits, i.e. 70 km/h?

Ans: Yes, your maximum speed was `70.65 km/h`.

Q11: How come the estimate in speed is so accurate while the estimate in position is not? This is very important! Recall what you find out about the GPS error in the first part of the exercise. Note that the speed is in this exercise calculated as the differance between two points divided by time (Q9).

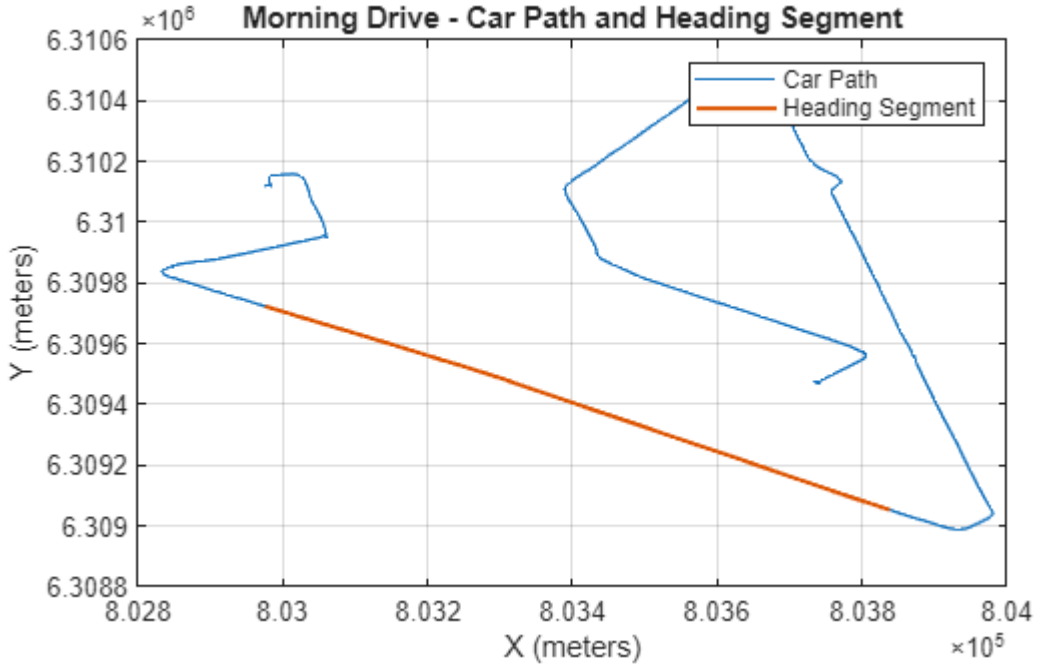
Ans: As you have stated, we calculate the speed as the difference between two point samples nex to each other (delta X and delta Y) and as we've seen in the auto-correlation plot, GPS errors are highly correlated between consecutive points so the common errors mostly cancels out while calculating dx or dy.

Plot the headings

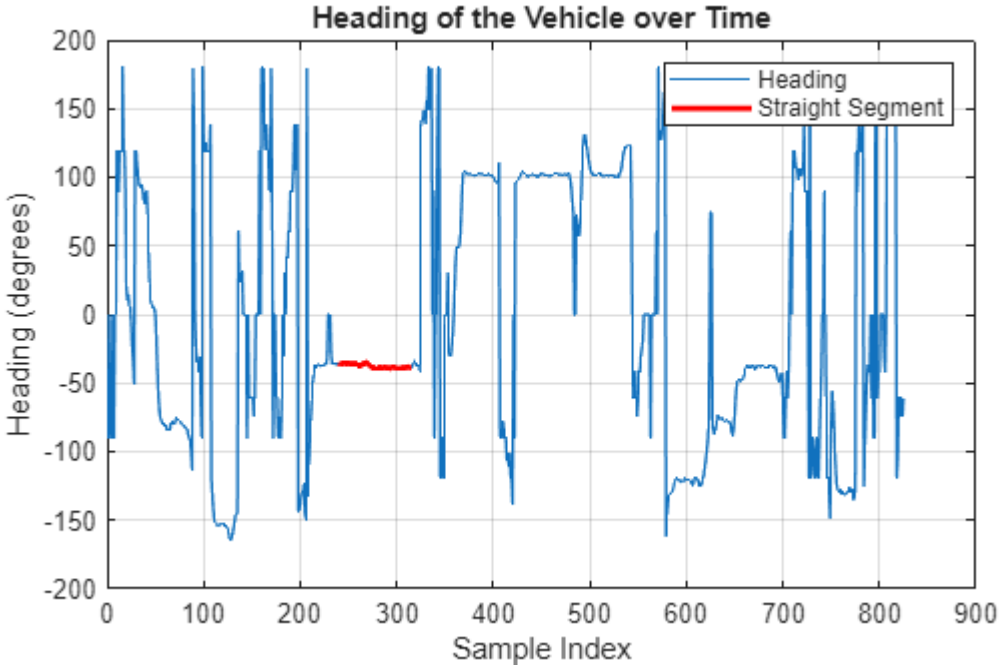
Calculate the headings (and plot them with respect to time) along the path (see compendium http://adamchukpa.mcgill.ca/web_ssm/web_GPS.html). Note that the the matlab function **atan2d()** can be used here instead of the "if statments" in the compendium.

Q12: Can you calculate the error in headings? Tips! If you only consider the headings along the straight-line path (along Laholmsvägen or Vrangelsleden) – can you then estimate the variance in the heading?

```
heading = atan2d(dy, dx); % returns -180 to 180
figure;
plot(D_X, D_Y); % full car path
hold on;
segment_idx = 240:315; % straight segment for heading variance
plot(D_X(segment_idx), D_Y(segment_idx), '-', 'MarkerSize', 4, 'LineWidth', 1.5);
% segment
title('Morning Drive - Car Path and Heading Segment');
xlabel('X (meters)');
ylabel('Y (meters)');
legend('Car Path','Heading Segment');
grid on;
```




```
%% Plot Heading over Time and Highlight Segment
figure;
plot(heading); % full heading signal
hold on;
heading_straight = heading(segment_idx);% segment used
plot(segment_idx, heading_straight, 'r', 'LineWidth', 2);% highlight segment
title('Heading of the Vehicle over Time');
xlabel('Sample Index');
ylabel('Heading (degrees)');
legend('Heading','Straight Segment');
grid on;
```



Mark the heading values used for calculating the heading variance in the heading plot. Also, mark the positions used for the heading calculation in the Morning drive plot above. Tips use the data points between 240:315.

```
mean_heading = mean(heading_straight);
heading_error = heading_straight - mean_heading;
% Variance and Standard Deviation
heading_variance = var(heading_error);
heading_std = std(heading_error);
fprintf('Heading variance (straight segment): %.4f deg^2\n', heading_variance);
```

```
Heading variance (straight segment): 2.4597 deg^2
```

```
fprintf('Heading standard deviation: %.4f deg\n', heading_std);
```

```
Heading standard deviation: 1.5683 deg
```

Submitt your exercise

The reports shall be handed in via Blackboard. All questions shall be answered properly, and elaborated figures shall be included. Submitt both the .mlx file and aslo save it as a PDF file, Submitt it as two separat files in Blackboard. Don't forgett to add your name(s) at the top of the document.

References

[1] W. Lechner and S. Baumann, Global Navigation Satellite Systems, Computers and Electronics in Agriculture, Volume 25, Issues 1-2, January 2000, Pages 67-85.

https://doi.org/10.1016/S0168-1699(99)00056-3

[2] Adamchuk, Viacheslav I., "EC01-157 Precision Agriculture: Untangling the GPS Data String" (2001). Historical Materials from University of Nebraska-Lincoln Extension. Paper 707.

https://www.researchgate.net/profile/Viacheslav_Adamchuk/publication/228382384_EC01-157_Precision_Agriculture_Untangling_the_GPS_Data_String/links/54ba7fbf0cf253b50e2d0234.pdf?origin=publication_list